PatchBlock in Front of a Vision-Language Agent: Detection, Recovery and Energy on Jetson Orin Nano

Fanuel Petros | Technical report (draft) | November 2026
An extension of PatchBlock (Chattopadhyay et al., DATE 2026) [1]

Summary. PatchBlock [1] is a CPU-side defense against adversarial patches that was evaluated on image classifiers and a detector, with energy reported for one laptop and only runtime reported for Jetson devices. This report places a reimplementation of PatchBlock in front of SmolVLM-500M, a small vision-language model choosing one of four actions, on a Jetson Orin Nano, and measures energy per decision in the 15W, 25W and MAXN SUPER power modes. Per-image targeted patches changed the action on 10 of 10 scenes. The SVD mitigation stage removed every attack when given the patch location, but the detection stage located patches at chance level, and smoothing randomly chosen chunks recovered as many scenes as PatchBlock did (best 4 of 10). The defense added 5.69 to 6.60 J per decision (+63% to +72%) and 0.80 to 1.01 s, consistent within 0.1 J over three runs; an estimated 84% to 92% of that energy is the board's baseline power during the extra time. MAXN SUPER was the most energy-efficient mode but triggered over-current events on the stock power supply.

1. Motivation and scope

PatchBlock [1] splits an image into chunks, flags statistically unusual chunks with an Isolation Forest [10] over mutual-information features, and replaces flagged chunks with a low-rank SVD reconstruction. It builds on ODDR [2] and earlier dimensionality-reduction defenses [3, 4]. Its evaluation leaves two gaps that this work addresses:

- **Model type.** It was tested on ResNet, VGG, ViT and YOLOv4 [1], not on a vision-language model driving an agent, a setting the eBRAIN survey [5] names as an open robustness problem.
- **Energy on the target hardware.** Joules were reported for a Lambda Tensorbook only; for Jetson Orin and Orin Nano the paper reports runtime, with no power-mode sweep.

This is an extension of the published defense, not a new method. Results describe this reimplementation, with the deviations listed in Section 3.2.

2. Setup

Item	Configuration
Hardware	NVIDIA Jetson Orin Nano (Seeed carrier), JetPack 6.2 (L4T R36.4.3), CUDA 12.6
Software	PyTorch 2.8.0, transformers 4.49.0, scikit-learn, NumPy 1.26.4
Model	SmolVLM-500M-Instruct [8], float32, greedy decoding, one 512 x 512 image tile
Agent output	One word from MOVE, STOP, FLAG, REPORT, constrained by the prompt
Scenes	10 COCO val2017 images [9], IDs 139, 285, 632, 724, 776, 785, 802, 872, 885, 1000
Power modes	15W, 25W, MAXN SUPER (nvpmodel 0, 1, 2); no 7W mode on this firmware

3. Method

3.1 Attack

For each scene, a centred square patch (153 x 153 px: 30% of the side, 9% of the area) was optimized directly in the model's 512 x 512 input, in the style of localized visible patches [6, 7]. The loss is the cross-entropy of the target word under teacher forcing; Adam, learning rate 0.02, 200 steps, model weights frozen. The target is STOP, or MOVE when the clean action is STOP. The clean action was read three times per scene; scenes that disagreed would have been excluded. Clean and patched readings use the same 512 x 512 image and preprocessing, so the patch is the only difference. The patches are white-box, per-image and digital: they are not universal, not printed, and were not optimized against the defense.

3.2 Defense reimplementation

Chunking: 50 px chunks (the paper's setting), an 11 x 11 grid with edge-aligned last row and column. Separating: four features per chunk, standardized, scored by an Isolation Forest (100 trees, 30% subsample, fixed seed); the most anomalous fraction of chunks is flagged. Mitigating: per-channel SVD of each flagged chunk, keeping 90% of the squared singular values. Deviations from the paper, where it was ambiguous or not reproducible:

- Standard Isolation Forest, not the paper's Fast Isolation Forest with gradient-based cuts.
- The paper describes mutual information both across colour channels and between neighbouring chunks; both are used (three inter-channel values, one neighbour value).

- The stride and the outlier threshold are not given unambiguously, so non-overlapping chunks are used and the flagged fraction is swept over 5%, 10%, 15% and 20%.

Three controls separate detection from mitigation: **oracle** smooths exactly the chunks overlapping the known patch (an upper bound, not a defense); **full-image SVD** applies the mitigation to the whole image, the simpler fallback in the lineage [4]; **random chunks** smooths the same number of chunks as PatchBlock, chosen at random (3 seeds per scene, 30 trials per fraction).

3.3 Energy measurement

tegrastats sampled the module input rail (VDD_IN, onboard INA3221) every 100 ms. In each power mode, each condition ran back-to-back decisions over the 10 scenes for at least 60 s; energy per decision is the time-weighted mean power times duration divided by decisions. Idle power was measured with the model loaded. The sweep was run three times with a different mode order each run. Every decision was checked against the action recorded in the attack and defense evaluations; GPU clock (sysfs), temperature and over-current event counters were logged.

4. Results

4.1 Attack

All 10 scenes were stable across the three clean readings. The targeted attack succeeded on **10 of 10** (clean actions: MOVE 5, REPORT 4, STOP 1). Crafting took 198.4 s per patch on average (197.4 to 201.0 s) at 25W, with 3.34 GB peak GPU memory.

4.2 Defense

Condition	Patched: recovered	Patched: attack holds	Clean: kept	CPU s / image
No defense	0/10	10/10	10/10	-
PatchBlock, 5% flagged	1/10	8/10	10/10	0.79
PatchBlock, 10% flagged	2/10	8/10	10/10	0.82
PatchBlock, 15% flagged	**4/10**	**4/10**	**10/10**	**0.84**
PatchBlock, 20% flagged	4/10	4/10	8/10	0.87
Oracle location (control)	7/10	0/10	7/10	0.10
Full-image SVD (control)	5/10	1/10	5/10	0.66

Table 1. Defense results on the 10 attacked scenes and their clean versions, 25W mode. "Recovered" means the action returned to the clean action.

- **Mitigation works when aimed correctly.** With the patch location given, the attack held on 0 of 10 scenes. The 3 unrecovered scenes (139, 724, 872) also changed action when the same region of the *clean* image was smoothed, so the loss of scene detail, not the patch, explains them.
- **Detection is at chance.** Chunk-level precision was 0.03 to 0.07 against a chance level of 0.066 (8 of 121 chunks are at least half covered by the patch), and recall did not exceed the flagged fraction.
- **Random chunks do as well or better** (Figure 1): 20%, 30%, 50% and 63% recovered against PatchBlock's 10%, 20%, 40% and 40%. Smoothing any part of these per-image patches tends to break them, so the recovery cannot be credited to the detection stage.
- **Full-image SVD is degenerate.** It returned MOVE for all 20 images, clean and patched; its 5 of 10 "recoveries" are the 5 scenes whose clean action is MOVE.

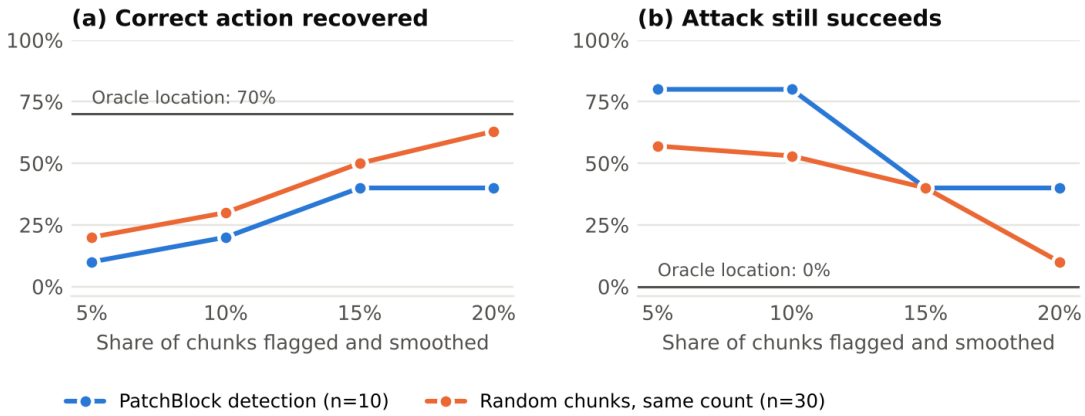


Figure 1. PatchBlock against random chunk selection at the same fraction. The horizontal line is the oracle control. PatchBlock: 10 scenes; random: 10 scenes x 3 seeds.

4.3 Energy and latency

Mode	Undefended (J)	Defended (J)	Overhead (J)	Overhead (%)	Latency (s)	PatchBlock CPU (s)
15W	9.32 ± 0.01	15.55 ± 0.09	6.23 ± 0.10	66.8 ± 1.1	0.80 → 1.69	0.77
25W	9.17 ± 0.04	15.76 ± 0.06	6.60 ± 0.04	72.0 ± 0.5	0.65 → 1.66	0.84
MAXN SUPER	9.06 ± 0.05	14.76 ± 0.05	5.69 ± 0.03	62.8 ± 0.5	0.56 → 1.36	0.65

Table 2. Energy per decision on attacked inputs, mean ± sample SD over 3 runs; idle power 5.98 W in every mode. Latency is the median per decision, undefended → defended. 1,758 decisions in total; every one matched its expected action.

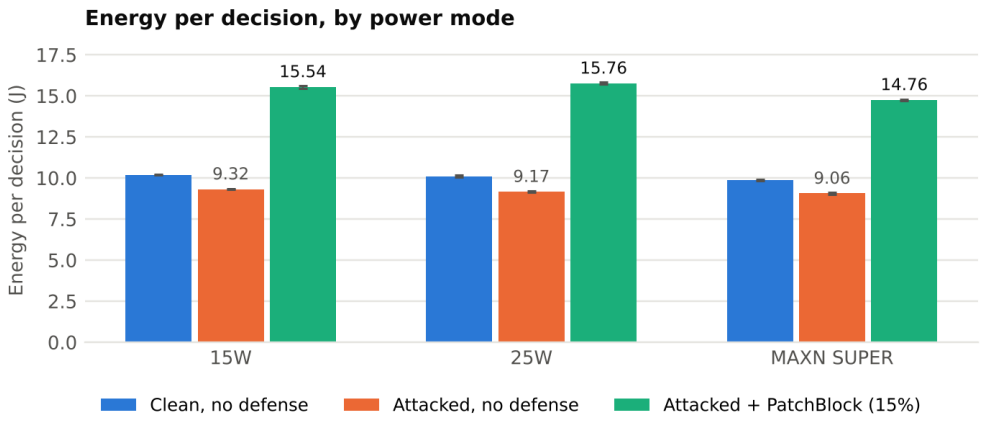


Figure 2. Energy per decision by condition and power mode. Error bars: SD over 3 runs (too small to see at this scale).

- **The defense adds about two-thirds to the energy and roughly doubles latency** in every mode. Run-to-run spread was at most 0.1 J, and runs in different mode orders agreed.
- **Most of the overhead is time, not computation.** Idle power multiplied by the extra latency accounts for an estimated 84% to 92% of the extra energy (Figure 3a; estimate from median latencies). A faster detection stage would remove most of the cost.
- **MAXN SUPER is the most efficient mode** for every condition: it draws the most power (about 15.8 W undefended) but finishes soonest. Undefended energy varies by only 3% across modes.
- **25W is the most expensive mode for the defended pipeline** because the PatchBlock CPU step was slower there (0.84 s) than at 15W (0.77 s) in every run. The cause was not identified; CPU clocks were not logged.
- **MAXN SUPER exceeded the stock supply's current limit**: 4 over-current events, all during undefended runs at 15 to 16 W, none in other modes. Results from the affected run matched the other runs, and the GPU clock minimum stayed at 816 MHz.

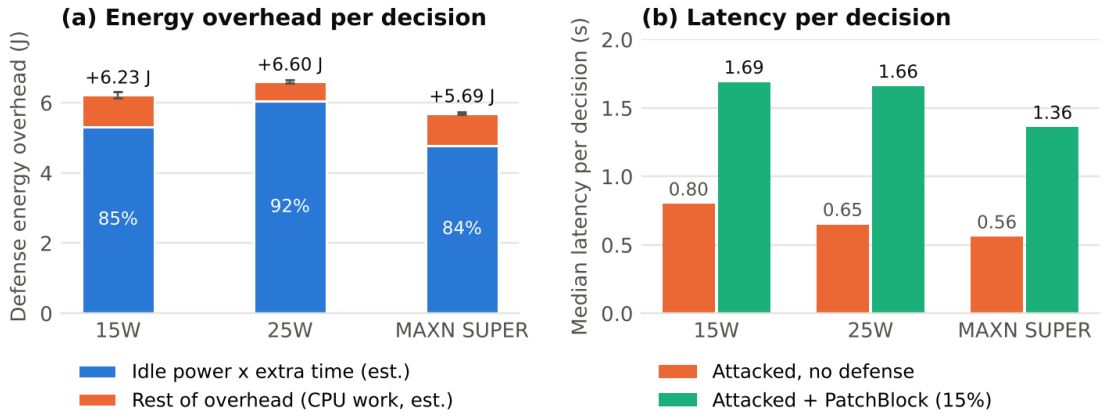


Figure 3. (a) Defense energy overhead per decision, split into idle power multiplied by the extra latency (estimate) and the remainder. (b) Median latency per decision.

5. Discussion

For a heavy model, the defense's relative cost is lower than for a CNN classifier. Here the PatchBlock CPU step took 1.0 to 1.3 times as long as a SmolVLM decision; in the paper it took roughly 2 to 4 times as long as the classifiers it protected [1]. The two implementations differ, so this comparison is indicative only. Because the CPU step is about as long as inference, running the defense in parallel with the GPU, as the paper proposes, would hide part of the latency

but make the defense the stage that sets throughput.

The defense result has a narrower reading than the energy result. Per-image patches are fragile, so smoothing almost any part of them breaks the attack; this makes them a weak test of detection. The finding is that, under these patches, PatchBlock's recovery cannot be attributed to its detection stage. It is not evidence that the published PatchBlock fails, and a robust patch is needed to test detection fairly.

6. Limitations

- Per-image, digital, white-box patches on 10 COCO scenes; no universal or physical patch, no camera frames, no defense-aware attacker.
- Reimplementation with the deviations in Section 3.2; CPU times are not comparable to the paper's.
- float32 inference; one board; the defense runs before inference, not in parallel with it.
- Clean and attacked energy differ partly because the answers differ in length ("REPORT." versus "STOP.", about 0.85 J), not only because of the attack.
- Remote-desktop software was running in the background with no viewer connected; the first energy run has no GPU clock data.

7. Next steps

- Craft a universal patch with Expectation over Transformation and repeat the defense and random-chunk comparison; then test it printed, through the camera.
- Implement the Fast Isolation Forest and a pipelined CPU/GPU version, and re-measure energy.
- Repeat the energy sweep in float16 and log CPU clocks to explain the 25W result.

References

[1] N. Chattopadhyay, A. Basit, A. Guesmi, M. A. Hanif, B. Ouni, M. Shafique, "PatchBlock: A Lightweight Defense Against Adversarial Patches for Embedded EdgeAI Devices," DATE 2026, arXiv:2601.00367.

[2] N. Chattopadhyay, A. Guesmi, M. A. Hanif, B. Ouni, M. Shafique, "ODDR: Outlier Detection & Dimension Reduction Based Defense Against Adversarial Patches," ICCV 2025.

[3] N. Chattopadhyay, A. Guesmi, M. Shafique, "Anomaly Unveiled: Securing Image Classification against Adversarial Patch Attacks," arXiv:2402.06249, 2024.

[4] N. Chattopadhyay, A. Guesmi, M. A. Hanif, B. Ouni, M. Shafique, "Defending against Adversarial Patches using Dimensionality Reduction," DAC 2024, arXiv:2311.12211.

[5] M. Shafique et al., "Adversarial and Backdoor Threats in Autonomous-Vehicle and Embodied-AI Systems," SOCC 2025.

[6] D. Karmon, D. Zoran, Y. Goldberg, "LaVAN: Localized and Visible Adversarial Noise," ICML 2018.

[7] T. B. Brown, D. Mane, A. Roy, M. Abadi, J. Gilmer, "Adversarial Patch," arXiv:1712.09665, 2017.

[8] Hugging Face, "SmolVLM-500M-Instruct," model card, HuggingFaceTB/SmolVLM-500M-Instruct.

[9] T.-Y. Lin et al., "Microsoft COCO: Common Objects in Context," ECCV 2014.

[10] F. T. Liu, K. M. Ting, Z.-H. Zhou, "Isolation Forest," ICDM 2008.